PHM NORTH AMERICA 2024 · DATA CHALLENGE

Helicopter Turbine Engine Health Estimation

A two-stage, physics-informed & probabilistic machine learning pipeline

AI in Industry — Project (6+3 CFU)

Kulsoom Zaidi · **Alina Baig**

DATASET

742,625 observations, zero missing values

742,625

sensor observations

7

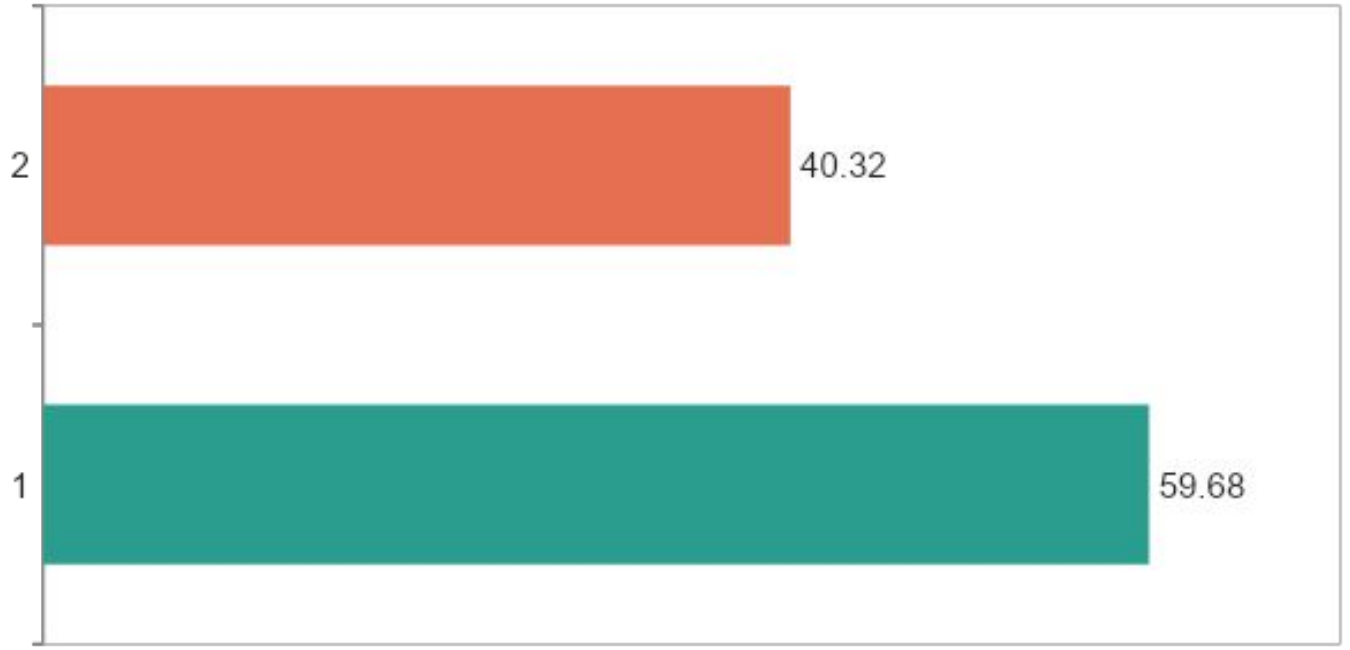
raw sensor features

0

missing values

Raw sensors: trq_measured, oat, mgt, pa, ias, np, ng

Class balance — faulty



Reasonably balanced — no aggressive resampling needed; stratified split preserves this ratio.

PROBLEM RECAP

One dataset, two predictions per engine

REGRESSION

trq_margin

Continuous torque-margin health indicator

CLASSIFICATION

faulty

Binary health state (0 = nominal, 1 = faulty) — with a confidence score

Our Architecture — 2-Stage, Physics-Informed + Probabilistic

Raw sensors

7 operating-condition readings

→

Stage 1 — Regression

Predict trq_margin
(+ uncertainty PDF)

→

Stage 2 — Classification

faulty label + confidence,
using Stage-1 output as a feature

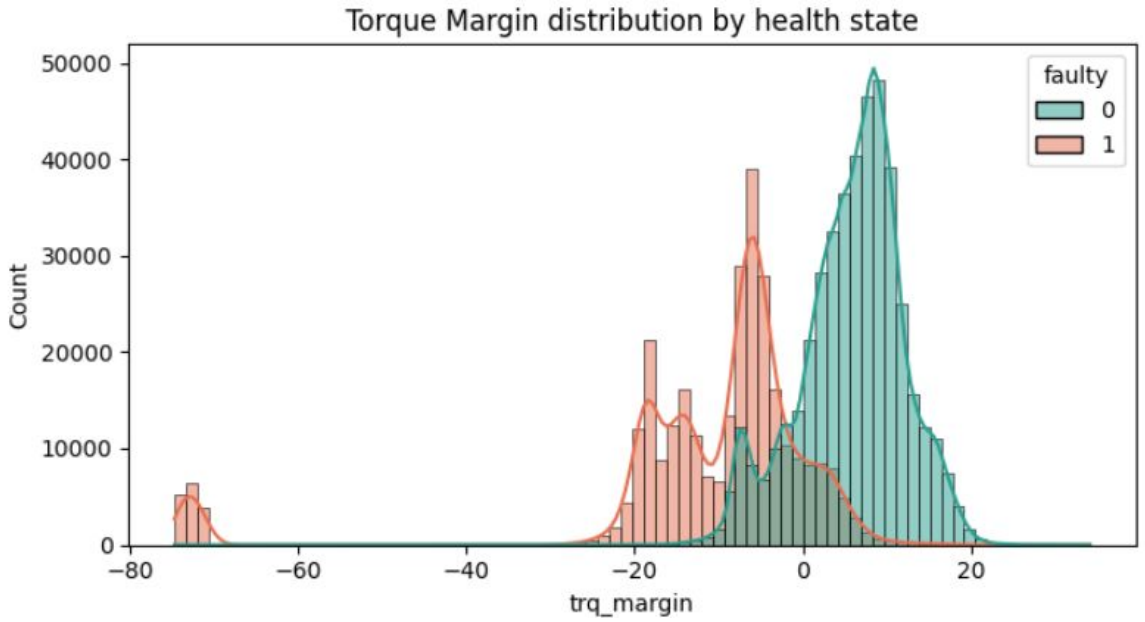
Why two stages? EDA shows trq_margin and faulty overlap significantly — a threshold rule alone caps out ~86% accuracy, so the classifier learns from the regression output instead of a hand-coded cutoff.

EXPLORATORY ANALYSIS

trq_margin separates classes — but overlaps

Key finding: Faulty engines show a wider, left-shifted trq_margin distribution, but the two classes overlap significantly. A simple threshold on trq_margin alone cannot cleanly separate them.

Torque margin distribution by health state (schematic)



Implication for the architecture

In the graph, we observed that the Torque Margin values of healthy and faulty engines overlap.

This means we cannot separate healthy and faulty engines using a single threshold value.

For example, some healthy engines and some faulty engines have very similar Torque Margin values.

Because of this overlap, a simple rule such as **"if Torque Margin is below a certain value, then the engine is faulty"** would make many incorrect predictions.

Therefore, instead of using a fixed cutoff value, we trained a dedicated machine learning classifier.

In our two-stage framework, the classifier does not use Torque Margin as the only decision factor.

FEATURE ENGINEERING

8 physics-informed features, built from 7 raw sensors

oat_mgt_diff

`mgt − oat`

Temperature rise across the engine — classic thermodynamic health indicator

trq_per_ng

`trq_measured / ng`

Torque normalized by gas-generator speed — engine efficiency proxy

ng_minus_np

`ng − np`

Speed mismatch between gas generator & power turbine — mechanical degradation signal

pa_per_np

`pa / np`

Available power normalized by net power — operating-regime indicator

mgt_per_pa

`mgt / pa`

Thermal loading per unit power

power_index

`np · ng / 100`

Combined power/speed interaction term

A/B test confirms the hypothesis: Raw only → MAE 0.639, R^2 0.9959 | Raw + engineered → MAE 0.592, R^2 0.9964

FEATURE SELECTION

Two complementary checks, cross-validated against each other

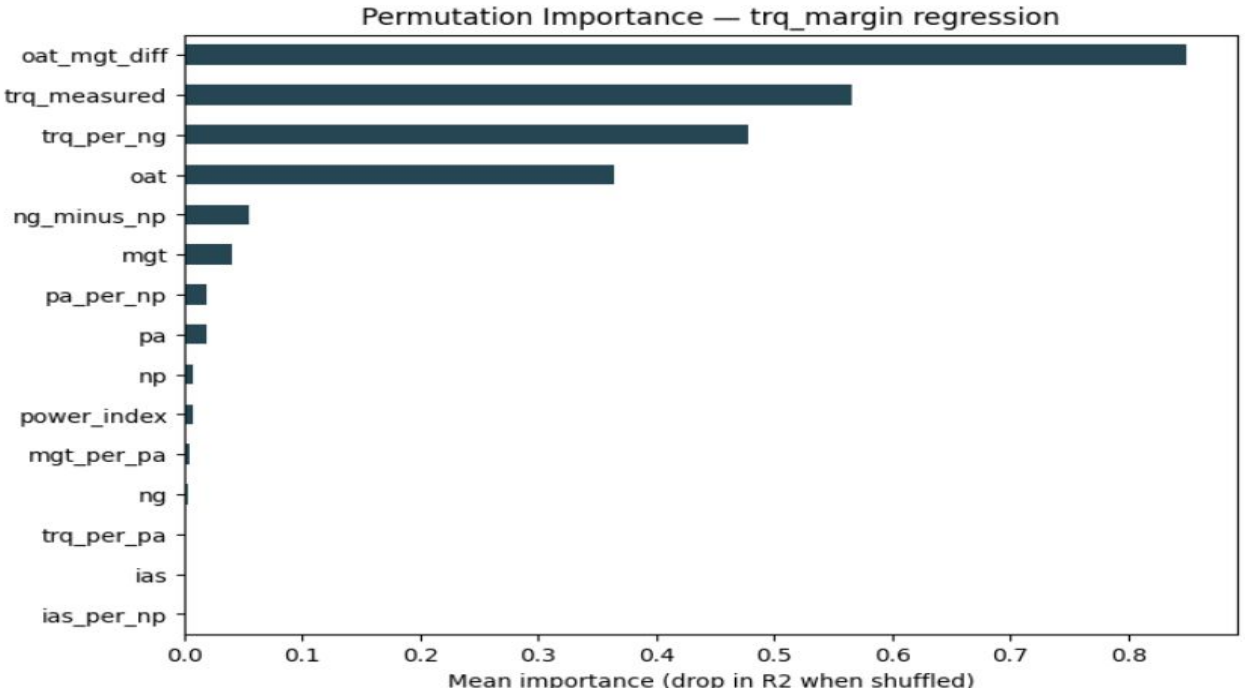
Spearman Correlation

Univariate, monotonic relationship with the target

catches non-linear monotonic effects

Permutation Importance

Real impact on a trained model's predictions — catches interactions correlation misses



Result

oat_mgt_diff and trq_per_ng win on both Spearman and permutation importance — confirming the physics reasoning.

power_index and ias_per_np scored low on both, so we drop them.

Final set: 13 features (7 raw + 6 engineered).

PROJECT SUMMARY

Why this architecture, in three sentences

1 Two stages, not one

faulty isn't a pure threshold on trq_margin — so we predict the continuous health signal first, then let the classifier use it as a feature.

2 Trees over neural nets

Dense tabular data, not sequential — gradient-boosted trees (LightGBM/XGBoost) consistently outperform plain MLPs here.

3 A distribution, not a point

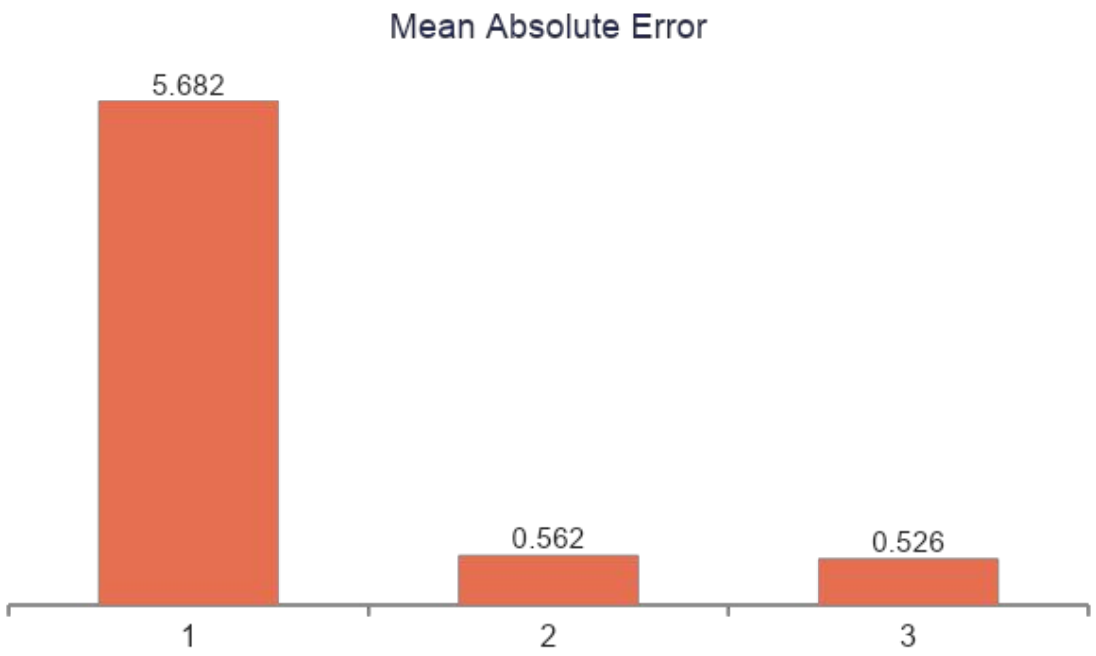
Quantile regression gives the center; 4 candidate families fit by BIC give the honestly-shaped spread (Student-t won).

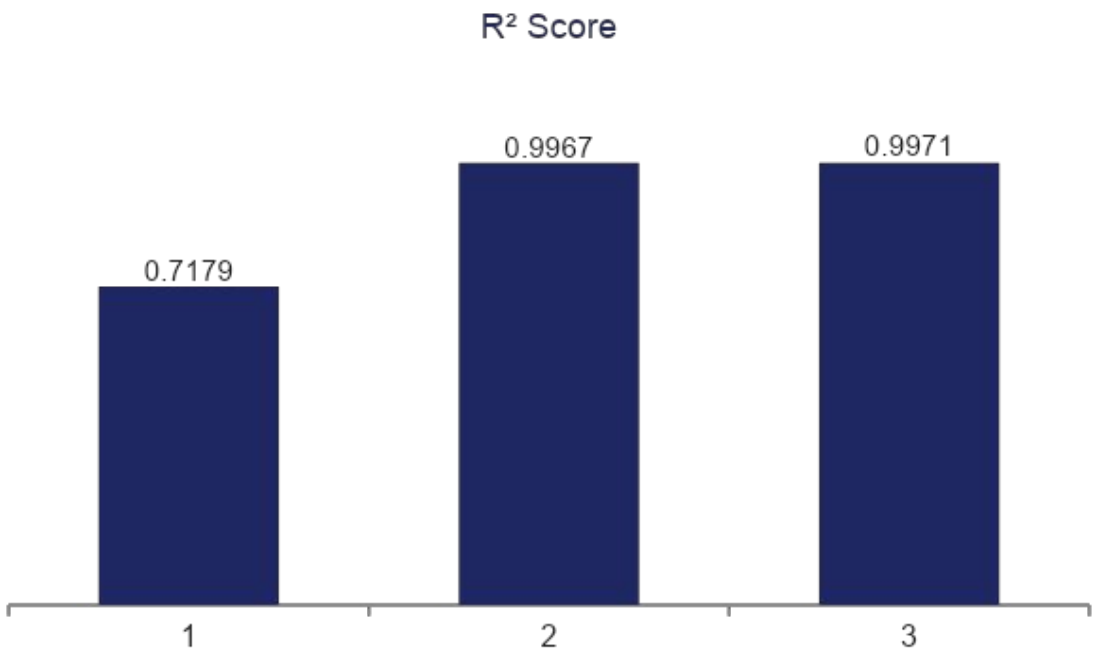
4 Calibrated to the scoring rule

Isotonic calibration + tuned shrinkage directly optimize the challenge's own confident-false-negative penalty.

STAGE 1 — REGRESSION

Predicting trq_margin: three models compared





Winner: XGBoost

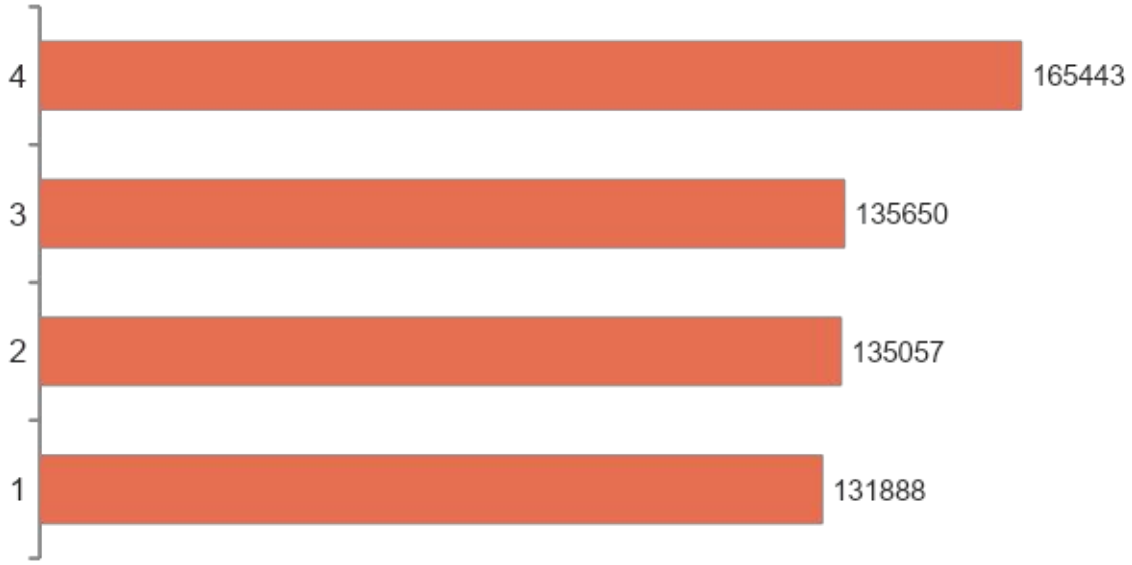
MAE = 0.526, RMSE = 0.756, R^2 = 0.997. Linear Regression trails badly (R^2 = 0.72) — proof the relationship is non-linear, justifying boosted trees over a simple baseline.

BEYOND A POINT ESTIMATE

Predicting a full distribution, not just a number

PHM2024 requires a probability distribution per prediction. We use quantile regression (10th / 50th / 90th percentile XGBoost models) for the center, then let the data pick the best-fitting spread — instead of assuming Normal.

Distribution fit — ranked by BIC (lower = better)



Student-t wins by a wide margin

Residuals are heavy-tailed: real data has more large surprises than a Normal distribution would predict — physically plausible, since faulty engines can have extreme torque margins. Assuming Normal would under-estimate the chance of extreme faults.

82.6%

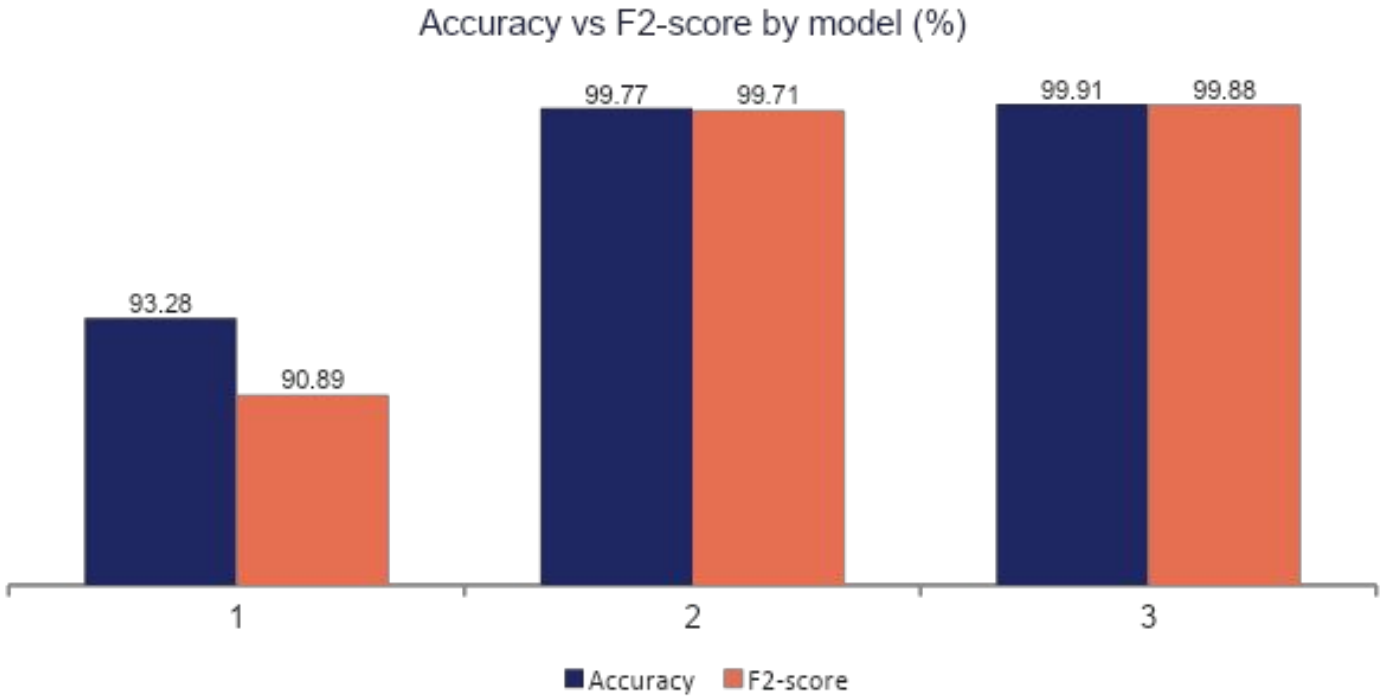
coverage of the predicted 10–90% interval (target: ~80%)

Well-calibrated uncertainty — the model isn't just confident, it's correctly confident.

STAGE 2 — CLASSIFICATION

faulty prediction: Stage-1 output feeds in as a feature

Key design choice: the classifier receives pred_trq_margin and its uncertainty (pred_std) from Stage 1, alongside the 13 sensor features — letting it use the regression's "expert opinion" without a hard-coded threshold.



XGBoost— winner (by F2)

Accuracy	**99.91%**
AUC	**0.99999**
F1-score	**99.89%**
F2-score	**99.88%**

5-fold CV confirms stability:

Acc 0.9991 ± 0.00003

AUC 0.99999 ± 0.00000

F2 0.9989 ± 0.00005

READING THE NUMBERS HONESTLY

99.9% accuracy — why, and what to watch for

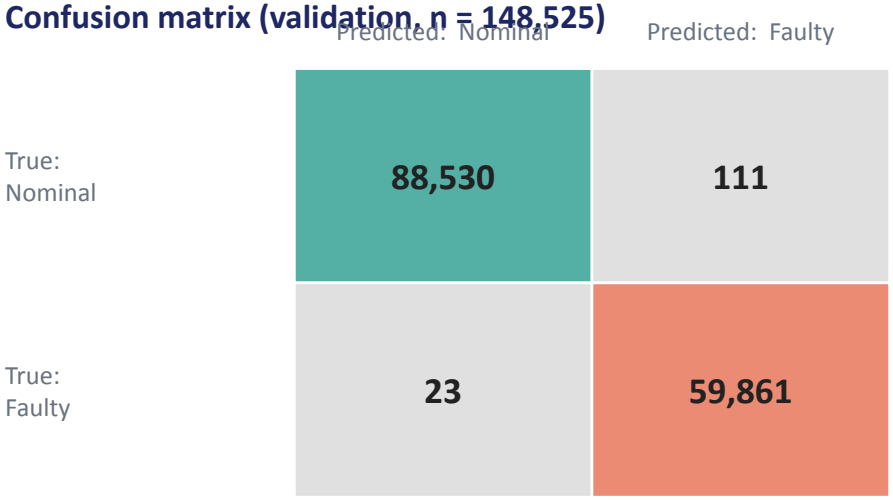
Why it's this high

trq_margin is an inherently strong signal for faulty (−0.61 correlation), and Stage 1 predicts it with $R^2 = 0.997$ — so pred_trq_margin gives Stage 2 an almost direct answer.

What to watch

Stage-1 features are computed in-sample here (mildly optimistic). A stricter pipeline uses out-of-fold predictions (cross_val_predict) so Stage 2 only ever sees Stage-1 outputs the way it would at true test time — a one-function change.

Confusion matrix (validation, n = 148,525)



Precision & recall ≈ 1.00 for both classes.

Only 134 misclassifications out of 148,525 validation rows.

CONFIDENCE CALIBRATION

Matching the challenge's own scoring function

The official scoring rule heavily penalizes confident false negatives (predicting "nominal" with high confidence when the engine is actually faulty). We calibrate probabilities with isotonic regression, then tune confidence shrinkage directly against that scoring function.

0.9972

Raw calibrated confidence score

0.6

Best shrinkage power found

0.9972

Best tuned challenge score

Step 1 — Isotonic calibration

CalibratedClassifierCV makes predict_proba mean actual P(faulty), not just a rank-ordered score.

Step 2 — Conservative shrinkage

Confidence is shrunk near the decision boundary, since the scoring function punishes over-confidence on the faulty side far more than under-confidence.

Step 3 — Grid search tuning

A small grid search over the shrinkage power picks the setting that maximizes the actual challenge score — a Bayesian-optimization-style tuning step.

EXPLAINABILITY

SHAP confirms the architecture works as designed

TreeExplainer (fast, exact for gradient-boosted trees) explains both stages — global feature attribution and local, single-prediction breakdowns.

1

Global — Stage 1

Summary/beeswarm plot ranks which sensor & engineered features move the trq_margin prediction most — oat_mgt_diff and trq_per_ng dominate, matching the permutation-importance findings.

2

Global — Stage 2

pred_trq_margin (the Stage-1 output) surfaces as a top-2 SHAP feature in the classifier — direct evidence the two-stage design is genuinely using the regression signal, not ignoring it.

3

Local — one prediction

A waterfall plot for a single engine shows exactly which sensor readings pushed that specific prediction up or down — the concrete example for a live defense.

END-TO-END PIPELINE

Raw sensor row(s) in → challenge-format JSON out

predict_for_submission(X_raw) returns exactly the PHM2024 schema:

```
{


  "class": 1,
  "class_conf": 0.94,
  "pdf_type": "t",
  "pdf_args": { "df": 1.81, "loc": 12.4, "scale": 0.50 }
}
```

- class / class_conf ← calibrated, shrinkage-tuned Stage-2 classifier
- pdf_type ← BIC-selected best-fit family (Student-t)
- pdf_args ← quantile-model median, shifted & scaled per row

Thank You

Questions & Discussion

PHM North America 2024 Data Challenge — Helicopter Turbine Engine Health Estimation

Kulsoom Zaidi · Alina Baig